

Strings In C

What Is a String?

⇒ A string is a sequence of characters stored in a character array.

⇒ The last character of a string is always the null character (`\0`), which indicates the end of the string.

Example:

```
char str[6] = "Hello"; // String: "Hello"
```

The array stores:

```
['H', 'e', 'l', 'l', 'o', '\0']
```

Declaring and Initializing Strings

Declaration : Declaring a string in C is as simple as declaring a one-dimensional array.

Syntax : `char string_name[size];`

Initialization : A string in C can be initialized in different ways.

- Initialize a string during declaration: `char str[6] = "Hello";`
- Assigning a String Literal without Size: `char str[] = "World";`
- The size of the array is automatically determined by the compiler.
- Initialize character by character : `char str[6] = {'H', 'e', 'l', 'l', 'o', '\0'};`

Declaring and Initializing Strings

Declaration : Declaring a string in C is as simple as declaring a one-dimensional array.

Syntax : `char string_name[size];`

Initialization : A string in C can be initialized in different ways.

- Initialize a string during declaration: `char str[6] = "Hello";`
- Assigning a String Literal without Size: `char str[] = "World";`
- Initialize character by character : `char str[6] = {'H', 'e', 'l', 'l', 'o', '\0'};`

Key point to emphasize

The last two forms look similar but behave differently:

- **a double-quoted literal ("Hello")** is auto-terminated by the compiler, while
- **a brace-enclosed character list ({'H','e',...})** is not — you must supply `'\0'` explicitly or the array is just a plain char array, not a valid C string.

Loop Through a String

```
#include <stdio.h>

int main (){

    char greeting[] = {'H', 'e', 'l', 'l', 'o', '\0'};
    for (int i = 0; i < 5; i++) {
        printf("%c", greeting[i]);
    }

    return 0;
}
```

Output : Hello

%c - Prints exactly one character. Used when you're pulling characters out one at a time.

Printing a String (Using %s Format Specifier)

```
#include <stdio.h>

int main (){

    char greetingOne[6] = "hello";
    char greeting[] = {'H', 'e', 'l', 'l', 'o', '\0'};
    printf("Greeting message One : %s\n", greetingOne);
    printf("Greeting message: %s\n", greeting );
    return 0;
}
```

Output :

Greeting message One : hello

Greeting message: Hello

%s prints an entire string in one call by internally scanning until it hits '\0'.

String Input Using scanf()

```
#include <stdio.h>

int main() {
    char name[20];
    printf("Enter your name: ");
    scanf("%s", name);
    printf(" your name is : %s ", name);
    return 0;
}
```

Output :
Enter your name: usha
your name is : usha

The "%s" format specifier is used in the scanf() statement.

String Input with Whitespace

`scanf("%s")` reads characters until it encounters a whitespace (space, tab, newline, etc.)

```
#include <stdio.h>

int main() {
    char greeting[20];
    printf("Enter a string:");
    scanf("%s", greeting);
    printf("You entered: ");
    printf("%s", greeting);
    return 0;
}
```

Output:

Enter a string: hello world
You entered: hello

String Functions in C

C provides a set of standard functions for string manipulation (declared in `<string.h>`).

Commonly Used Functions:

Function	Description	Example
<code>strlen(str)</code>	Returns the length of the string (excluding <code>\0</code>).	<code>strlen("Hello") → 5</code>
<code>strcpy(dest, src)</code>	Copies the string <code>src</code> to <code>dest</code> .	<code>strcpy(dest, "World")</code>
<code>strncpy(dest, src, n)</code>	Copies up to <code>n</code> characters from <code>src</code> to <code>dest</code> .	<code>strncpy(dest, "World", 3) → "Wor"</code>
<code>strcat(dest, src)</code>	Appends <code>src</code> to the end of <code>dest</code> .	<code>strcat("Hello", "World") → "HelloWorld"</code>
<code>strcmp(s1, s2)</code>	Compares two strings (0 if equal, <0 if <code>s1 < s2</code> , >0 if <code>s1 > s2</code>).	<code>strcmp("abc", "abd") → -1</code>
<code>strchr(str, c)</code>	Finds the first occurrence of character <code>c</code> in the string <code>str</code> .	<code>strchr("Hello", 'e') → "ello"</code>

Example: Read String Input Using fgets() Function

The fgets() function can be used to accept input from any input stream, such as stdin (keyboard) or FILE (file stream).

Syntax: fgets(char arr[], size, stream);

```
#include <stdio.h>

int main() {
    char name[20];
    printf("Enter a string:\n");
    fgets(name, sizeof(name), stdin);
    printf("You entered: \n");
    printf("%s", name);
    return 0;
}
```

Output :
Enter a string:
welcome to cdac
You entered:
welcome to cdac

fgets stops only at a newline (not at a space).

Plain gets() is unsafe/removed from modern C and fgets() is its replacement.

Printing String Using fputs() Functions

```
#include <stdio.h>

int main (){

    char name[20] = "Rakesh Sharma";
    printf("With puts(): \n");
    fgets(name,sizeof(name),stdin);
    printf("With fputs(): \n");
    fputs(name, stdout);
    return 0;

}
```

Output:
With puts():
Rakesh Sharma
With fputs():
Rakesh Sharma

fputs(name, stdout) then prints the string to standard output.

String Length

- The length of a string is the **number of characters before the null character**.
- Use the **strlen** function from **<string.h>**

Example:

```
#include <stdio.h>
#include <string.h>
int main() {
    char alphabet[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    printf("%d", strlen(alphabet));
    return 0;
}
```

Output: Length of string: 26

strlen() counts only the visible characters and stops counting at '\0', so it returns 26

Concatenate Strings

To concatenate (combine) two strings, you can use the `strcat()` function:

Example

```
char str1[20] = "Hello ";  
char str2[] = "World!";  
// Concatenate str2 to str1 (result is stored in str1)  
strcat(str1, str2);  
// Print str1  
printf("%s", str1);
```

The **strcat(dest, src)** function copies the string `src` to the end of `dest`. It replaces the old null terminator of `dest` with the first character of `src`, and it adds a new null terminator at the very end of the combined string.

Copy Strings

To copy the value of one string to another, you can use the **strcpy()** function:

Example

```
char str1[20] = "Hello World!";
```

```
char str2[20];
```

```
// Copy str1 to str2
```

```
strcpy(str2, str1);
```

```
// Print str2
```

```
printf("%s", str2);
```

strcpy(dest, src) copies every character of src into dest, including the terminating '\0'.

Compare Strings

To compare two strings, you can use the `strcmp()` function.

```
char str1[] = "Hello";
```

```
char str2[] = "Hello";
```

```
char str3[] = "Hi";
```

```
// Compare str1 and str2, and print the result
```

```
printf("%d\n", strcmp(str1, str2)); // Returns 0 (the strings  
are equal)
```

```
// Compare str1 and str3, and print the result
```

```
printf("%d\n", strcmp(str1, str3)); // Returns -4 (the strings  
are not equal)
```

`strcmp` compares two strings character by character using their ASCII codes. `str1` and `str2` are identical, so `strcmp` returns 0 (zero).

How Strings Are Stored in Memory

Strings are stored as arrays in contiguous memory locations.

Each character occupies one byte, and the null character (`\0`) marks the end.

Example:

```
char str[6] = "Hello";
```

Memory layout:

Address	Value
---------	-------

1000	'H'
------	-----

1001	'e'
------	-----

1002	'l'
------	-----

1003	'l'
------	-----

1004	'o'
------	-----

1005	'\0'
------	------

Declare and Initialize an 2DArray of Strings

Syntax : `char strings [no_of_strings] [max_size_of_each_string];`

Example:

```
char strings[3][10] = {"Hello", "World", "C"};
```

This creates a 2D array with 3 strings, each of up to 9 characters (10th space for \0).

Declare and Initialize an 2DArray of Strings

Example: `char strings[3][10] = {"Hello", "World", "C"};`

Print each character in a new line

```
#include <stdio.h>
```

```
int main() {  
    char strings[3][10] = {"Hello", "World", "C"};  
  
    // Loop through each string  
    for (int i = 0; i < 3; i++) {  
        // Loop through each character until null terminator  
        for (int j = 0; strings[i][j] != '\0'; j++) {  
            printf("%c\n", strings[i][j]); // print each character on new line  
        }  
    }  
  
    return 0;  
}
```

Accessing Strings

Use indexing to access individual strings and characters.

Example:

```
char strings[3][10] = {"Welcome", "To", "CDAC"};
```

```
printf("%s\n", strings[0]);
```

```
printf("%c\n", strings[2][2]);
```

strings[0] -> Output: Welcome

strings[2][2] -> Output: A

Printing An Array of Strings

Example : Traversing an Array of Strings

```
#include <stdio.h>

int main (){

    char langs [10][15] = {

        "PYTHON", "JAVASCRIPT", "PHP",

        "NODE JS", "HTML", "KOTLIN", "C++",

        "REACT JS", "RUST", "VBSCRIPT"

    };

    for (int i = 0; i < 10; i++){

        printf("%s\n", langs[i]);

    }

    return 0;

}
```

Output:

PYTHON
JAVASCRIPT
PHP
NODE JS
HTML
KOTLIN
C++
REACT JS
RUST
VBSCRIPT

Problem statement 1

Write a C program that fulfills the following requirements:

1. **Input:** The program should accept a string of characters.
2. **Vowel Array:** Define a character array containing both lowercase and uppercase vowels ('a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U').
3. **Processing:** Iterate through each character of the input string and compare it against the characters in the vowel array.
4. **Output:** Print each character from the input string that is a vowel, followed by a space.

Solution 1

```
#include <stdio.h>

int main() {
    char str[100]; // Declare a character array to store the user's input string
    char vowels[] = {'a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'};

    printf("Enter a string: ");
    scanf("%[^\n]", str); // Read the entire line of input, including spaces

    printf("Vowels in the string: ");

    for (int i = 0; str[i] != '\0'; i++) {
        for (int j = 0; j < 10; j++) {
            if (str[i] == vowels[j]) {
                printf("%c ", str[i]);
                break;
            }
        }
    }

    printf("\n");
    return 0;
}
```

Solution 1

Comparison: Why Use It?

Imagine the user types Hello World into the console.

Code Template	Resulting Value in <code>str</code>	Reason
<code>scanf("%s", str);</code>	"Hello"	Stops reading immediately at the first space.
<code>scanf("%[^\\n]", str);</code>	"Hello World"	Reads the space and only stops when you hit Enter .

Problem statement 2

WAP a C program that accepts a string, then prints each character one by one while also identifying whether it's a lowercase letter, an uppercase letter, or a digit or a special character.

Sample Input and Output:

Input: Hello!123

Output:

```
Enter a string: Hello!123
```

```
Analyzing each character:
```

```
Character 'H' is an uppercase letter.
```

```
Character 'e' is a lowercase letter.
```

```
Character 'l' is a lowercase letter.
```

```
Character 'l' is a lowercase letter.
```

```
Character 'o' is a lowercase letter.
```

```
Character '!' is a special character or symbol.
```

```
Character '1' is a digit.
```

```
Character '2' is a digit.
```

```
Character '3' is a digit.
```

Solution 2

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int main() {
    char str[100];

    printf("Enter a string: ");
    scanf("%s", str);

    printf("\nAnalyzing each character:\n");

    for (int i = 0; i < strlen(str); i++) {
        char ch = str[i];

        printf("Character '%c' is ", ch);

        if (islower(ch)) {
            printf("a lowercase letter.\n");
        } else if (isupper(ch)) {
            printf("an uppercase letter.\n");
        } else if (isdigit(ch)) {
            printf("a digit.\n");
        } else if (strchr("!@#$%^&*() +-=[]{};:\",<.>./", ch) != NULL) {
            printf("a special character.\n");
        } else {
            printf("an unrecognized character.\n");
        }
    }

    return 0;
}
```

Problem statement 3

Problem Statement: Simple C Login System

Develop a C program that simulates a basic user login system. The program should perform the following tasks:

Register a User: Prompt the user to set a username and a password. Store these credentials in character arrays.

Login: After registration, prompt the user to log in by entering their username and password.

Validate Credentials: Compare the entered login credentials with the stored registration credentials.

Display Outcome: Print a success message if the credentials match or a failure message if they do not.

Solution 3

```
#include <stdio.h>
#include <string.h>

int main() {
    char registeredUsername[50];
    char registeredPassword[50];
    char loginUsername[50];
    char loginPassword[50];

    // 1. User Registration
    printf("--- User Registration ---\n");
    printf("Create a username: ");
    scanf("%s", registeredUsername);

    printf("Create a password: ");
    scanf("%s", registeredPassword);

    printf("\nRegistration successful! Your account is ready.\n\n");

    // 2. User Login
    printf("--- User Login ---\n");
    printf("Enter your username: ");
    scanf("%s", loginUsername);

    printf("Enter your password: ");
    scanf("%s", loginPassword);

    // 3. Validate Credentials
    // strcmp returns 0 if the strings are identical.
    if (strcmp(loginUsername, registeredUsername) == 0 && strcmp(loginPassword,
registeredPassword) == 0) {
        printf("\nLogin successful! Welcome, %s.\n", registeredUsername);
    } else {
        printf("\nLogin failed. Invalid username or password.\n");
    }

    return 0;
}
```

Solution 3

```
--- User Registration ---
```

```
Create a username: usha
```

```
Create a password: usha@123
```

```
Registration successful! Your account is ready.
```

```
--- User Login ---
```

```
Enter your username: usha
```

```
Enter your password: usha@123
```

```
Login successful! Welcome, usha.
```

```
PS D:\HPC - CDAC\C> █
```

Solution 3

```
--- User Registration ---
```

```
Create a username: usha
```

```
Create a password: usha@123
```

```
Registration successful! Your account is ready.
```

```
--- User Login ---
```

```
Enter your username: usha
```

```
Enter your password: usha
```

```
Login failed. Invalid username or password.
```

```
PS D:\HPC - CDAC\C>
```

Problem statement 4

Problem Statement: Secure Password Creator

Create a C program that acts as a secure password creator. The program must repeatedly prompt the user to enter a new password until it meets all of the following security criteria:

Length: The password must be at least 8 characters long.

Uppercase: It must contain at least one uppercase letter (A-Z).

Lowercase: It must contain at least one lowercase letter (a-z).

Digit: It must contain at least one digit (0-9).

Special Character: It must contain at least one special character from the set !@#\$%^&*()_+~[]{}|;:'",.<>/?.

For each attempt that fails to meet the criteria, the program should provide specific feedback to the user, listing exactly which requirements were not met. Upon successful creation, the program should confirm that the password has been set.

Solution 4

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>

int main() {

    char password[100];

    int has_upper, has_lower, has_digit, has_special;

    printf("Please set a new password that meets the following criteria:\n");

    printf("- At least 8 characters long\n");

    printf("- At least one uppercase letter\n");

    printf("- At least one lowercase letter\n");

    printf("- At least one digit\n");

    printf("- At least one special character from !@#... etc.\n\n");

    while (1) {

        has_upper = has_lower = has_digit = has_special = 0;

        printf("Enter your new password: ");

        scanf("%s", password);

        // Check password criteria

        int length = strlen(password);

        for (int i = 0; i < length; i++) {

            if (isupper(password[i])) {

                has_upper = 1;

            } else if (islower(password[i])) {

                has_lower = 1;

            } else if (isdigit(password[i])) {

                has_digit = 1;

            } else if (strchr("!@#%^&*()_+~[]{};:'\"|<,.>?/", password[i]) != NULL) {

                has_special = 1;

            }

        }

    }
```

```
// Check for all criteria and provide specific feedback
if (length < 8) {
    printf("Error: Password must be at least 8 characters long.\n");
}
if (!has_upper) {
    printf("Error: Password must contain at least one uppercase
letter.\n");
}
if (!has_lower) {
    printf("Error: Password must contain at least one lowercase
letter.\n");
}
if (!has_digit) {
    printf("Error: Password must contain at least one digit.\n");
}
if (!has_special) {
    printf("Error: Password must contain at least one special
character.\n");
}

// If all criteria are met, exit the loop
if (length >= 8 && has_upper && has_lower && has_digit &&
has_special) {
    printf("\nPassword set successfully! All criteria were met.\n");
    break;
}

printf("\nPlease try again.\n\n");
}

return 0;
}
```

Problem statement 4

Secure User Authentication

Create a C program that implements a simple user authentication system. The system should first guide a user through a **registration process** where they must create a username and a password that meets specific security criteria.

The password criteria are as follows:

- Minimum **8 characters** in length.

- Contains at least one **uppercase letter**.

- Contains at least one **lowercase letter**.

- Contains at least one **digit**.

- Contains at least one **special character** from the set !@#\$%^&*()_+~[]{};:'"<,>./.

The program should use a loop to repeatedly prompt the user for a password until all criteria are met. Once the password is valid, the program should proceed to a **login process**, where it prompts the user to enter the previously set username and password.

Finally, the program must validate the entered login credentials against the stored registration data and display a message indicating whether the login was successful or failed.

Note: The solution should use string-handling functions from the <string.h> and <ctype.h> libraries to perform the necessary checks.

Solution 4

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char registeredUsername[50];
    char registeredPassword[50];
    char loginUsername[50];
    char loginPassword[50];

    // User Registration
    printf("--- User Registration ---\n");
    printf("Create a username: ");
    scanf("%s", registeredUsername);

    int password_is_valid = 0;
    while (!password_is_valid) {
        int has_upper = 0, has_lower = 0, has_digit = 0,
        has_special = 0;

        printf("Create a password (min 8 chars, with upper, lower,
        digit, and special char): ");
        scanf("%s", registeredPassword);

        int length = strlen(registeredPassword);

        // Check password criteria
        if (length >= 8) {
            for (int i = 0; i < length; i++) {
                if (isupper(registeredPassword[i])) {
                    has_upper = 1;
                } else if (islower(registeredPassword[i])) {
                    has_lower = 1;
                } else if (isdigit(registeredPassword[i])) {
                    has_digit = 1;
                } else if (strchr("!@#%&*()_+=[{};<,>.?/'",
                registeredPassword[i]) != NULL) {
                    has_special = 1;
                }
            }
        }
    }
```

```
// Check if all flags are set
if (length >= 8 && has_upper && has_lower && has_digit &&
has_special) {
    password_is_valid = 1;
    printf("\nPassword set successfully! All criteria were met.\n");
} else {
    printf("Password is too weak. Please try again.\n");
}

printf("\nRegistration successful! Your account is ready.\n\n");

// User Login
printf("--- User Login ---\n");
printf("Enter your username: ");
scanf("%s", loginUsername);

printf("Enter your password: ");
scanf("%s", loginPassword);

// Validate Credentials
if (strcmp(loginUsername, registeredUsername) == 0 &&
strcmp(loginPassword, registeredPassword) == 0) {
    printf("\nLogin successful! Welcome, %s.\n",
    registeredUsername);
} else {
    printf("\nLogin failed. Invalid username or password.\n");
}

return 0;
}
```